Sujet du TP 1 de INFO731

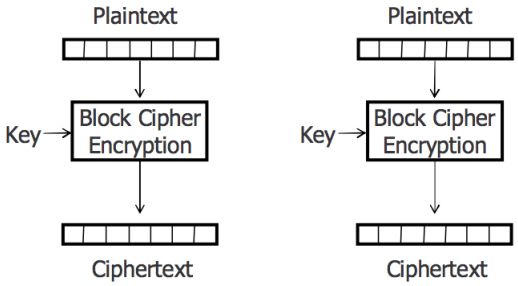


Figure 1: Electronic Code Book (ECB) Mode

Block Cipher modes of Operation

A mode of operation is a way of using block ciphers like AES, which only encrypt a fixed number of bits, to encrypt arbitrary-length messages. You can experiment with AES and modes of operation in Python (with PyCryptodome library)

```
THIS_MODE = AES.MODE_ECB
KEY_SIZE = AES.key_size[0]
key = Random.new().read(KEY_SIZE)
iv = Random.new().read(AES.block_size)
message = 'hello world'
cipher = AES.new(key, THIS_MODE, iv)
ciphertext = cipher.encrypt(pad(message))
```

We suggest you use the following pad and unpad functions to pad your message with 0's to the appropriate length before encrypting, and remove the zeros upon decryption:

```
def pad(s):
"""Takes in a string, and returns that string with zeros appended.
Padding should be performed prior to encryption.
"""
return s + b"\0" * (AES.block_size - len(s) % AES.block_size)
def unpad(s):
"""Removes trailing zeros from a string.
Unpadding should be performed after decryption.
"""
return s.rstrip(b »\0")
```

1- Electronic Code Book (ECB) Mode

Electronic Code Book (ECB) mode is illustrated in Figure 1. ECB mode essentially encrypts each block of the ciphertext independently.

2- In tux6.ppm, you will find an image. (PPM binary format is a simple image format where the pixels were represented as uncompressed bytes.) Encrypt that image using Electronic Code Book

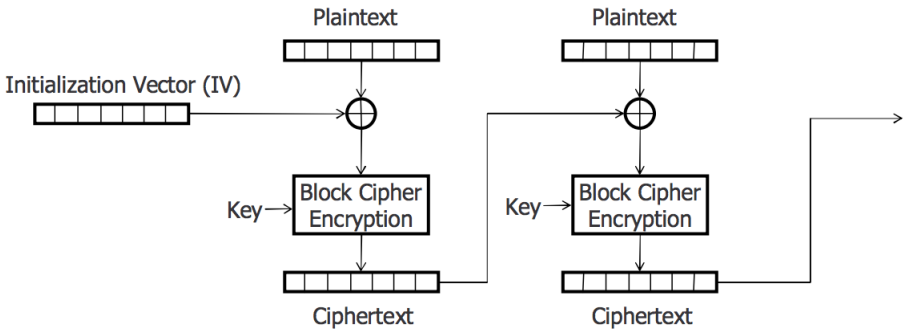
(ECB) mode. Encrypt the same image using Cipher Block Chaining (CBC) mode (described part 3). (When encrypting, remove the three header lines from tux6.ppm, and prepend them unencrypted to the ciphertext.)

The definition of CPA security for a symmetric encryption scheme. When playing the following game with a Challenger, no efficient Adversary should be able to win with probability much more than half:

- A key k for the encryption scheme is chosen uniformly at random by the Challenger.
- The Adversary, which does not know k, is given access to an oracle that computes $Enc_k(\cdot)$ on a message m of the adversary's choice.
- The Adversary chooses two messages of equal length, m_0 and m_1, and sends them to the Challenger.
- The Challenger chooses a random bit b, produces the challenge ciphertext $c^* = Enc_k(m_b)$, and sends c^* to the adversary.
- The Adversary may continue to send any message of its choice to the the oracle that computes $Enc_k(\cdot)$.
- The Adversary outputs $b' = 0$ if it thinks that c^* was the encryption of m_0, and $b' = 1$ otherwise.
- The Adversary wins if $b' = b$.

Prove that AES in ECB mode cannot satisfy the definition of CPA-secure encryption. In other words, present an algorithm for an Adversary that wins the game described above.

3- Cipher Block Chaining (CBC) Mode



Cipher Block Chaining mode is illustrated above. It XORs each ciphertext block with the next plaintext before encryption. For the first block, a random initialization vector (IV) is used instead. Note that the IV needs to be included as part of the ciphertext in order for decryption to be possible. It has been proven that CBC mode, unlike ECB mode, is CPA-secure if the block cipher encryption function is a pseudorandom permutation.

However, if the IVs are incremented like a counter instead of being chosen at random, this breaks down, as you will show in this section.

In CBC_ciphertext you will find a ciphertext. You know that it was produced using the initialization vector in CBC_iv1, and that the ciphertext encrypts the message in CBC_message1: “turn to page 01". The sender isn’t using CBC mode correctly; they should be choosing random initialization vectors, but instead, they are incrementing the initialization vector like a counter, adding 1 to it for every message sent. A few messages later, they will be using the IV in CBC_iv2. What two messages can you specify as m 0 and m 1 in the CPA security game when it is time to use CBC_iv2, such that you will be able to tell the difference between the two encryptions?

4-Modes of Encryption in the Wild
In Google chrome, you can see details about the security of a website by clicking on the lock icon to the left of the URL. If the site is secure, it should say “Secure Connection - Your information (for example, passwords or credit card numbers) is private when it is sent to this site.” If you click on “Details" right under that statement, you will see some additional information, like what mode of operation is being used in TLS.

Choose 3 websites that offer secure connections and use the AES block cipher. What mode of operation are they using?

Getting Familiar with RSA

Here, we summarize a naive version of the RSA (“textbook RSA") encryption and digital signatures schemes. This version of RSA should not be used; you will show in this lab several ways in which it can be vulnerable. Consult https://en.wikipedia.org/wiki/RSA_(cryptosystem) for more detail.

Encryption

Below we describe the key generation (KeyGen), encryption (Enc), and decryption (Dec) algorithms of RSA.

1. KeyGen:
   (a) Choose two random primes p and q.
   (b) Let $n = pq$ be the modulus.
   (c) Select a random encryption exponent e such that the greatest common denominator of e and $(p-1)(q-1)$ is 1 (that is, e and $(p-1)(q-1)$ are relatively prime).
   (d) Let the decryption exponent d be $e^{-1} \bmod (p-1)(q-1)$.
   (e) Return the public key $pk = (n,e)$ and the secret key $sk = d$.
2. Enc$(pk = (n,e), m)$, where m is a message to be encrypted:
   (a) Return the ciphertext $c = m^e \bmod n$.
3. Dec$(sk = d, c)$, where c is a ciphertext to be decrypted:
   (a) Return the recovered message $m = c^d \bmod n$.

As an example, say that during KeyGen, I choose p = 5 and q = 11. Therefore, I have n = 55. If I choose e = 3, I need to find a d such that 3d mod (5–1)(11–1) = 3d mod 40 = 1. I can try a few numbers, and see that 3×27 mod 40 = 81 mod 40 = 1, so I set d = 27. 1 Now, say that you know only n = 55 and e = 3. Encryption is just raising the message m to the power of e; you can encrypt m = 51 as c = 51^3 mod 55 = 132651 mod 55 = 46.

Decryption is the inverse of encryption; it takes the e-th root of the ciphertext. e and d are chosen in such a way that raising to the power of d is the same as taking the eth root. Therefore, I, who also know d = 27, can decrypt c = 46 by taking m = 46^{27} mod 55 = 51.

Taking the e-th root of a value modulo an n whose factorization you don't know is considered to be unrealistic for large numbers. However, in our example, someone else, who doesn't know d = 27, can also figure out what m is, just by trying each number modulo 55 and raising it to e = 3. (If they get 46, they know they've found the right m!) This is only possible because we chose small p and q. If p and q are over 1000 bits long, instead of just 5 bits long, decryption without knowledge of d (or the factorization of n) becomes unrealistic.

1. $\text{Sig}(sk = d, m)$, where m is a message to be signed:
   (a) Return the signature $\sigma = m^d \bmod n$.
2. $\text{Ver}(m, \sigma, pk = (n, e))$, where m is a message and σ is a signature to be verified:
   (a) Verify that $\sigma^e \bmod n = m \bmod n$.

3-RSA in Python

You can experiment with RSA yourself, using Python. In tools.py, we provide several functions you can use in this exploration. For instance, we provide text to integer (and integer to text) conversion, because the RSA algorithms assume that the message m is an integer modulo n.

• text_to_int: takes in a string and returns an integer which can be used in RSA.

• int_to_text: takes in an integer and returns the corresponding string.

We also provide a few other functions:

• find_root: takes in integers x and r, and returns the integer component of the rth root of x. (Python's built in pow function can be used to do this, but it doesn't work for very large 'long'-type values.) √ For instance, find_root(x = 50,r = 2) = 7. 50 ~ 7.07106, and 7 is the integer part of 7.07106.

• combine_moduli: takes in two integer moduli n 1 and n 2 and two integers x 1 and x 2 , and returns an integer x mod n 1 n 2 such that x mod n 1 = x 1 and x mod n 2 = x 2 .

For instance, combine_moduli(n 1 = 5,n 2 = 7,x 1 = 2,x 2 = 3) = 17, since 17 mod 5 = 2, and 17 mod 7 = 3.

The file part1.py shows how to use Python to do some basic RSA operations:
We can also use math operations in Python to explore RSA ciphertexts and signatures manually. For instance, you can manually verify the signature. The modulus can be accessed as public_key.n, and the public exponent can be accessed as public_key.e.

print "%0128x" % pow(signature[0], public_key.e, public_key.n) # The suffix of the signature^e mod n should be:

print SHA256.new(message).hexdigest()

4-Small Message Space Attack

Imagine that you are a government agent trying to determine when your neighboring country, Malland, will attack. Open part2_ctext to find a "textbook RSA" ciphertext sent by Malland to its ally, Horridland. You know that the Mallanders are likely to be saying one of three things:

1. "attack tomorrow at dawn",

2. "attack just before dusk", or

3. "attack early next week".

The file part2_key.pub contains Horridland’s public key. Use this knowledge to figure out what the message is.

What would you do to prevent Malland from executing the same attack on ciphertexts you send?